

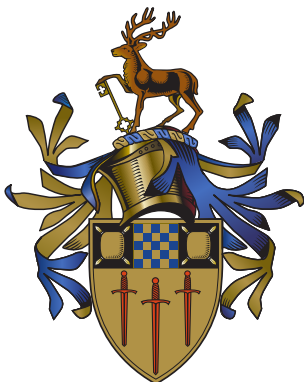
Investigating Ambisonic Game Audio Engine Architecture

Micah Strange

Spatial Audio Experiment
(Experiment conducted with Leon Paterson-Stephens)

Institute of Sound Recording
University of Surrey

May 2023



Contents

Chapter 1: Introduction	1
Chapter 2: Ambisonics in Video Games	2
Chapter 3: Experiment	4
3.1 Experiment Aims	4
3.2 Audio Engine Design	4
3.3 Interfacing with Unity	7
3.4 Ambisonic Encoder	8
3.5 Ambisonic Rotator	8
3.6 Ambisonic Transformer	9
3.7 Ambisonic to Binaural Decoding	9
3.8 Demo Scene	9
Chapter 4: Evaluation	11
4.1 Efficiency	11
4.2 Ease of Use	11
4.3 Quality of Output	11
4.4 Future Work	12
Chapter 5: Conclusion	13
References	14

Chapter 1

Introduction

Localisation of diegetic sound sources is an important feature in many genres of video games. Stereo audio is commonly used, providing left/right localisation [Broderick et al. 2018]. Furthermore, modelling of occlusion and reverb can provide cues for distance and position of objects. However, front/back and up/down localisation is more difficult to simulate. Audio engines can achieve this through a number of spatial techniques, such as binaural rendering over headphones or panning across loudspeakers in a surround sound format. The increasing use of virtual reality has further required games to have accurate localisation.

However, a problem is presented when designing spatial audio engines as a number of consumer output formats are required, such as binaural over headphones, 5.1 and 7.1 surround [Goodwin 2019: C.19]. It is impractical to provide separate mixes for all formats. A solution is to use an intermediate format that contains all the necessary spatial and mix information and then decode to each individual output.

Ambisonics is a 3-dimensional audio format based on representation of the soundfield as spherical harmonics [Malham 1999]. Originally using 4 channels, ambisonics are often stored in 'B-format', consisting of an omnidirectional channel and 3 perpendicular figure-of-eight channels [Furse 2009]. "Higher Order Ambisonics (HOA)" allows increasing localisation detail with increasing channels. This format has been implemented in a number of games such as the Colin McRae Rally racing series [Horsburgh et al. 2011]. Commonly, second and third order ambisonics are used or hybrid methods such as third order for horizontal components and first order for vertical components [Goodwin 2019: p.217].

This report will explore the use of 2nd order ambisonics to allow audio engines to be designed and used for multiple output formats. Chapter 2 will explore reasons for using ambisonic engines. Chapter 3 will describe an experiment implementing ambisonics in an audio engine. Chapter 4 will discuss and evaluate the findings and suggest further improvements. Chapter 5 concludes the report.

Chapter 2

Ambisonics in Video Games

Ambisonics contains full sphere spatial information, and is therefore suitable to use as a universal storage format [Goodwin 2019: C.19]. It is commonly implemented as an intermediate master mix bus, which can then be passed to an appropriate decoder dependant on the endpoint format. Sources that are chosen to be spatial are encoded to ambisonics and added to the bus. These could be mono voices that have a 3D position in the game, or a pre-rendered ambisonic recording used as ambience. Non spatial sounds can still be used, as they can be directly routed to the appropriate output channels .

Figure 2.1 shows a basic overview of how different voices are routed depending on their type.

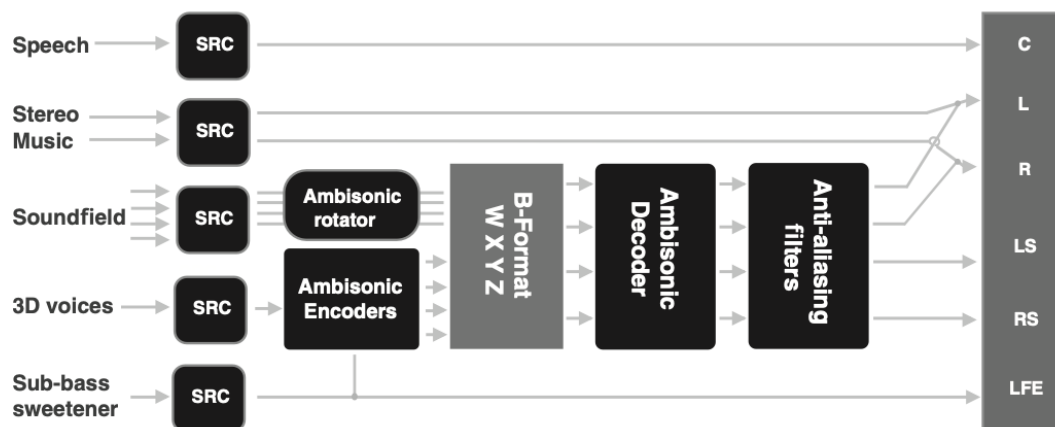


Figure 2.1: Basic signal flow for an ambisonic workflow with a 5.1 endpoint. From [Goodwin 2019: p.154]

There are a number of benefits to designing the engine in this way:

- It can be written and used without specific endpoints in mind [Goodwin 2019: p.199-201]. As it also provides a way of mixing, it prevents issues that may occur from folding-down large surround layouts. The level of spatial accuracy is still known as it is dependant on the order of ambisonics used.
- The modular approach to the decoder allows enhancement specific to the platform and output format, as large speaker arrangements can have complex decoding when the exact position of speakers is known [Furse 2009]. Improved decoders can be implemented afterwards, making the system future-proof [Goodwin 2019: p.201-202]. Ambisonics can also scale down easily for use on less powerful devices by using low order implementations.

- Rotation and manipulation of ambisonics is relatively simple and can be performed more smoothly than conventional pairwise panning between speakers [Goodwin 2019: p.198]. Encoding mono sources with source spread control is also achievable [Gorzel et al. 2019].
- When rendering ambisonics binaurally, A fixed number of Head-Related Transfer Functions (HRTFs) are used dependant on the rendering method and not individual audio sources. Therefore increasing the number of objects in a scene does not affect the HRTF calculations. For a given CPU load, more objects can be mixed and there is more flexibility adding objects and pre-mixed soundfields [Goodwin 2019: p.202].

Chapter 3

Experiment

3.1 Experiment Aims

To investigate the use of ambisonics, a game audio engine was created. This was achieved in C++ using the JUCE framework [Raw Material Software Limited 2023] to handle IO and audio processing. The engine interfaces with Unity [Unity Technologies 2023], a popular video game engine.

The primary aim was to investigate how the architecture of an audio engine is affected by the use of ambisonics. The interaction between components and design of the engine as a whole was considered to assess the benefits of an intermediate ambisonic bus. The individual modules were not fully optimised, but by assessing the quality of a basic implementation the architecture as a whole can be considered. The success of the engine was informally assessed by considering its efficiency, ease of use and output spatial quality.

A further aim was to investigate the relationship between different source types. Some audio such as music and menu sound effects should not be spatialised. 3D voices that have a location in the game world need encoding to ambisonics.

Pre-mixed ambisonic audio could be treated in multiple ways. When used as ambience, they should not be linked to a location but may rotate according to listener head movements.

Ambisonic audio may also be linked to a location in the game world. However, consideration is needed to change the audio as the listener moves around and rotates their head. This experiment proposes an implementation to handle this change.

3.2 Audio Engine Design

Second order ambisonics was chosen as HOA are commonly used. Binaural reproduction over headphones was chosen as the output for this experiment as it is a commonly used and allows further investigation into HRTFs and efficiency of binaurally rendering multiple sounds.

Horsburgh et al. [2011] outlined a method of categorising sound types needed in a game. Based on this and the aim of investigating the relationship between source types, the engine was designed to have a number of input types:

- **World Space Objects:** These are diegetic sounds that the character can see and interact

with such as foley sound effects and character voices. These are likely to be mono audio files but could also be multichannel, with ambisonic objects having a given size. All world space objects have a coordinate point associated to them.

- **Ambient:** Pre-mixed soundfields that have no specific location attached to them. However, they rotate based on player movement. These could be ambisonic recordings of ambience. In this experiment, these sources are always 2nd order B-format ambisonic files.
- **Non-spatial sounds:** This is for audio such as stereo music and menu sound effects. These have no spatial information and therefore are not included in the intermediate ambisonic bus and can be passed directly to the appropriate output based on the endpoint used.

Figure 3.1 shows an overview of the audio engine. The remainder of this chapter will discuss the individual modules and explain their design.

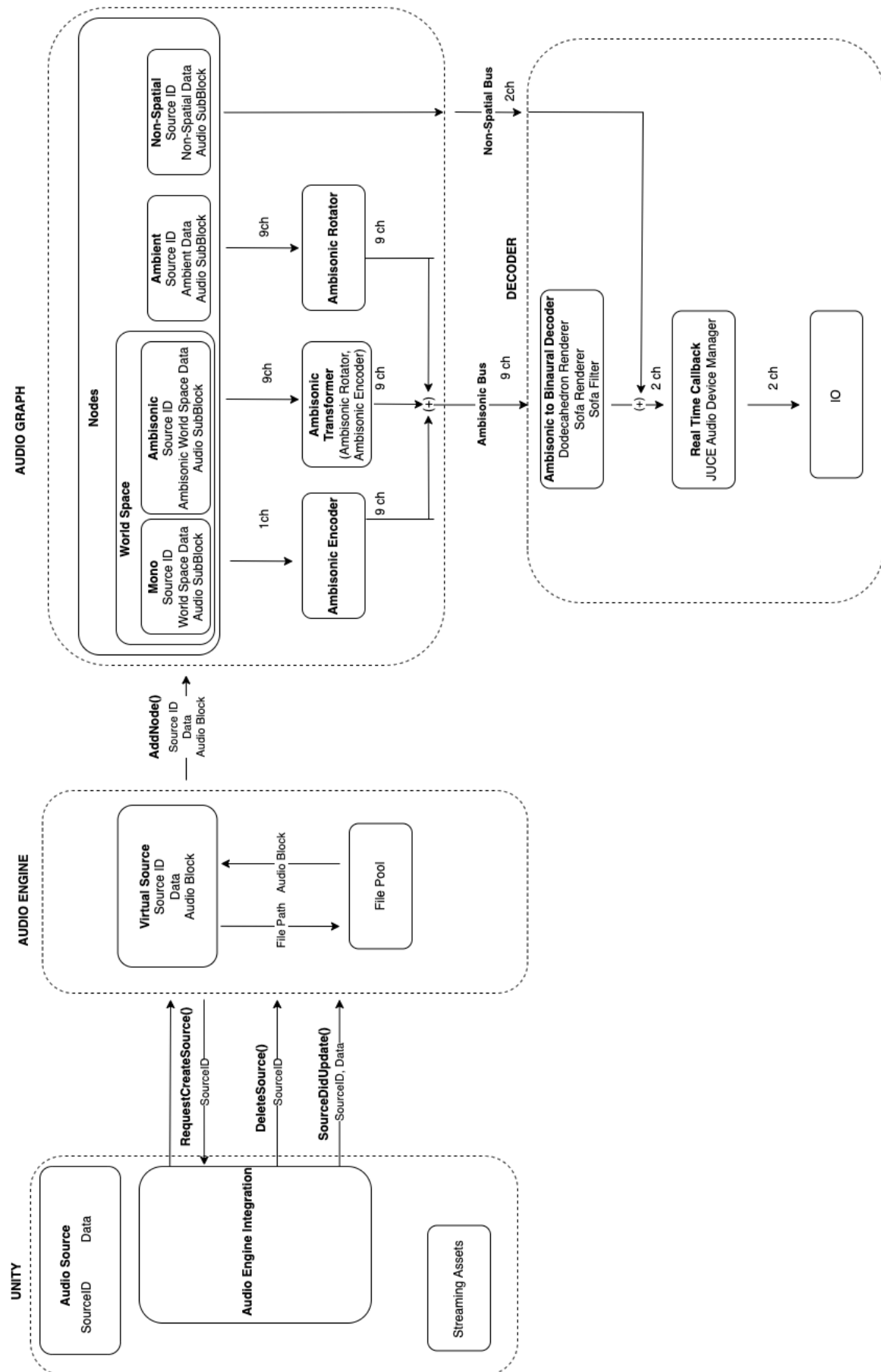


Figure 3.1: An overview of the audio engine.

3.3 Interfacing with Unity

Objects in Unity can have an *Audio Source* component attached to them. When created, a request is sent to the audio engine for a unique ID. Parameters are stored that contain the necessary spatial information. When a change occurs to the object, a message is sent to the audio engine with the new parameters:

```
SourceDidUpdate(  
  
    string sourceID,  
  
    bool isPlaying,  
  
    float volume,  
  
    string filePath,  
  
    Vector3 relativePosition,  
  
    enum sourceType,  
  
    Vector2 relativeRotation,  
  
    float radius  
  
)
```

IsPlaying and *filePath* are used to tell the audio engine what and when to play a sound, with a *volume*. *relativePosition* are Cartesian coordinates sent relative to the listener as these are used to determine the distance based attenuation and spatial position of world space objects.

sourceType indicates whether the source is world space mono, world space ambisonic, ambient or non-spatial. *relativeRotation* is used for ambient sources and world space ambisonics when the listener is inside the *radius*. Figure 3.2 shows the *Audio Source* component in Unity, where the appropriate parameters can be changed.

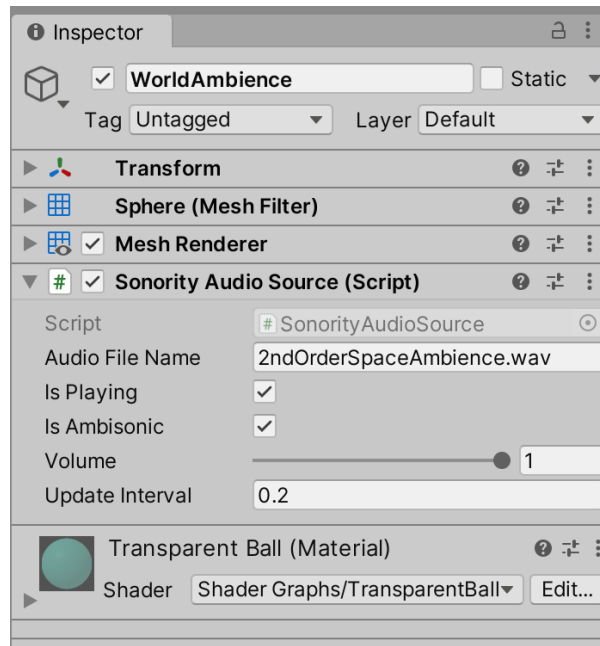


Figure 3.2: An object in Unity with the *Audio Source* component attached

3.4 Ambisonic Encoder

The audio block of a source is passed to the encoder along with its Cartesian coordinates, which are converted to spherical coordinates. Encoding a mono source to the ambisonic bus can be achieved through addition of the input on each channel multiplied by coefficients. The azimuth and elevation are used to calculate the coefficients of each channel, as described by Malham [1999], whilst the distance given by the spherical coordinates is used to calculate distance-based attenuation. The output channels are iterated through, adding the input data multiplied by the coefficients and distance attenuation directly to the ambisonic bus.

It was found that the objects were located appropriately at large distances but would be not be realistic when close or inside the listener as their position would change location too sensitively depending on head movement. Goodwin [2019: p.209] provides a solution for this by introducing a size to the source that changes with distance. As the distance decreases, the W component increases so that the source becomes monophonic.

3.5 Ambisonic Rotator

Rotating ambisonics was achieved by rotating around the Z axis ("Yaw") using the azimuth and around the Y axis ("Pitch") using the elevation [Blue Ripple Sound 2023]. A further improvement would include rotation around the X axis ("Roll"), when the listener tilts their head. It is also possible to further improve the efficiency of HOA rotations using recursion [Gorzel et al. 2019].

3.6 Ambisonic Transformer

The structure allows this type of source to be processed differently to the mono world space and ambient sources. This hybrid structure uses both the spherical coordinates and listener rotation data.

World space ambisonic sources are treated as ambient sources when the player is inside its radius as only rotation is applied. When the player is outside the radius, the W channel is passed to the ambisonic encoder and is treated as a mono source. To smooth this transition, a basic cross fade is applied between the full ambisonic buffer and the encoded mono buffer when the player is just outside of the object's radius.

3.7 Ambisonic to Binaural Decoding

The audio engine implements a dodecahedron virtual loudspeaker array to decode the second order ambisonic bus. This arrangement is commonly chosen for 2nd order as it is linearly spaced and has sufficient outputs to present horizontal and vertical information [Gorzel et al. 2019; Armstrong et al. 2018].

The decoder class retrieves the Head Related Impulse Responses (HRIRs) for each virtual loudspeaker position by reading a Spatially Oriented Format for Acoustics (SOFA) file [Majdak et al. 2013]. The file was read using the libmysofa library [Hoene 2023]. Before processing begins, these filters and delays are applied to an array of 12 HRTF renderers. The position of the loudspeakers does not change as rotation is applied to the ambisonic bus. A combination of the ambisonic channels are then given to each renderer which convolves the input with the HRIRs and adds delays to the left and right channels. The matrix used for the coefficients of each channel was taken from the ambisonic configuration files found in the SADIE database [Armstrong et al. 2018].

It was found that this decoder worked sufficiently in the engine. Further improvements to the decoder such as personalised HRTFs could improve performance, as suggested by Zhang et al. [2021]. However, it may be beneficial to reduce the accuracy of the decoder to improve efficiency, such as through symmetrical head assumptions [Gorzel et al. 2019].

3.8 Demo Scene

To informally assess the experiment a demo scene was created to show how the audio engine links to objects in Unity. A screenshot can be seen in Figure 3.3. The green objects are world space objects. The large sphere is an ambisonic world space object. When the player is inside the sphere, a full ambisonic render is heard. As the player moves away, the object becomes mono.

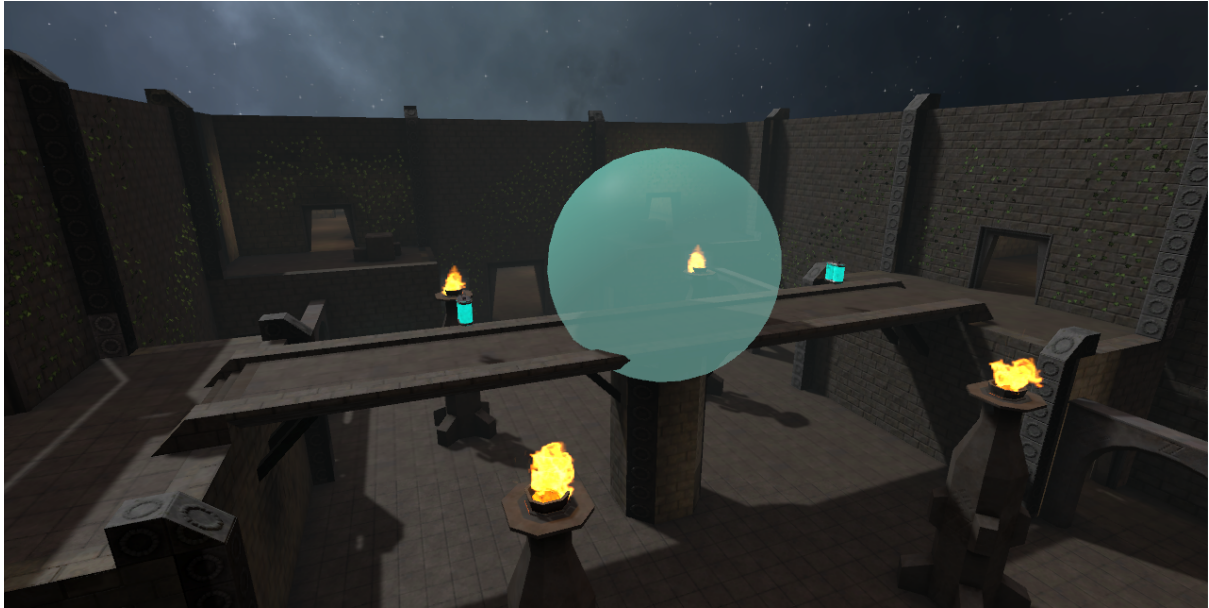


Figure 3.3: A screenshot of the demo scene in Unity. The green objects have an Audio Source component attached

Chapter 4

Evaluation

4.1 Efficiency

A large benefit to an ambisonic engine is the fixed number of HRTFs. This implementation required 12 HRTF renderers regardless of the number of objects. Furthermore, the HRIRs applied to each virtual loudspeaker remains constant as rotation is applied to the ambisonic signals.

As a large number of sources may be present in a scene, it is important for the encoder and transformer modules to be efficient. Gorzel et al. [2019] suggests utilising the symmetrical properties of spherical harmonics to reduce the number of calculations. A look up table can be used to store the spherical harmonic coefficients for 0-90° at 1° resolution. Furthermore, Single Instruction Multiple Data (SIMD) processor instructions could be used to add sources to the ambisonic buffer simultaneously.

4.2 Ease of Use

The method of sending messages between Unity and the audio engine was successful as only small amounts of spatial information is needed. Adding an ambisonic object only required the type to be specified.

The use of `std::unordered_map` meant that virtual sources in the audio engine and audio players in the graph could be added by creating an object with an Audio Source component in Unity.

All world-space and ambience sources are mixed onto one ambisonic bus. This is then passed to the decoder with non-spatial sounds, which are routed to the appropriate outputs. In the case of a binaural endpoint, these sounds are added to the left and right outputs as normal.

4.3 Quality of Output

The demo scene allowed the quality of the binaural output to be assessed. For mono objects, these were spatialised well, with basic distance attenuation vastly improving the localisation. However, as discussed in section 3.4, interior panning would improve this. This is easily achievable with the engine design.

Ambisonic world space objects were found to be only partially successful. Inside the object, rotation was successful and when the listener was distant it was plausible as a mono source. However, when close to the object, there was incorrect perceived width. This is because the width should gradually collapse to mono as the listener walks away. This could be implemented in a similar way to the "Spread" parameter in Wwise [Audio Kinetic 2023]. This would allow the listener to walk around the object and have it be localisable on the correct side of their head.

4.4 Future Work

There are many other aspects to a game audio engine that have not been discussed in this report, such as resource management and applying audio effects.

Reverb buses are also an important factor in the spatialisation of world space objects. A possible implementation could use an aux send to an effects bus, which is then mixed with the ambisonic bus, as used by Furse [2009].

As discussed previously, the modular nature of the engine allows improvements to individual components such as the encoders and decoders.

The manipulation of world space ambisonic sources could be improved. A further benefit of ambisonics is the ability to manipulate in a number of ways such as warping, widening and directional loudness [Kronlachner and Zotter 2014]. These transformations could be used to further interact with world space ambisonic sources.

Chapter 5

Conclusion

This report has investigated the use of ambisonics in video game audio engines. With an increase in spatial audio in games, a number of approaches can be taken, such as mixing in Dolby Atmos or HRTF rendering over headphones [Broderick et al. 2018]. However, ambisonics provides a flexible universal format for mixing allowing a variety of headphone and loudspeaker arrangements to be used.

An experiment was conducted by creating an audio engine that uses an intermediate 2nd order ambisonic bus. The results showed that sufficient localisation was achieved with mono and ambisonic objects, whilst being more efficient than individually rendering each object binaurally. The modular design allows versatility and scalability depending on the desired endpoint, which is suitable when designing games.

The relationship between pre-mixed ambisonic audio and diegetic voices in the game world was also explored. Rotating the audio when inside an ambisonic object and treating the source as a mono voice when distant was tested as a possible implementation. However, it was found that further manipulation of the source width is needed to improve plausibility.

A main drawback of this experiment is the basic design of the engine. Further improvements would be needed to scale the engine so that is suitable for complex game design. Furthermore, the addition of reverb sends and filtering would increase the quality of spatialised sound.

Bibliography

Armstrong, C., Thresh, L., Murphy, D., and Kearney, G. (2018): ‘A Perceptual Evaluation of Individual and Non-Individual HRTFs: A Case Study of the SADIE II Database’, *Applied Sciences*, 8(11), p.2029, doi:10.3390/app8112029.

Audio Kinetic (2023): ‘Effect of Spread’, URL https://www.audiokinetic.com/en/library/edge/?source=Help&id=effect_of_spread. (Accessed: 26 May 2023).

Blue Ripple Sound (2023): ‘HOA TECHNICAL NOTES - B-FORMAT ROTATION’, URL <http://www.blueripplesound.com/notes/rotations>. (Accessed: 26 May 2023).

Broderick, J., Duggan, J., and Redfern, S. (2018): ‘The Importance of Spatial Audio in Modern Games and Virtual Environments’, in *2018 IEEE Games, Entertainment, Media Conference (GEM)*, pp.1–9, doi:10.1109/GEM.2018.8516445.

Furse, R. W. (2009): ‘Building an OpenAL Implementation Using Ambisonics’, in *Audio Engineering Society Conference: 35th International Conference: Audio for Games*, London, UK, 11-13 February: Audio Engineering Society. Paper Number: 20.

Goodwin, S. N. (2019): *Beep to Boom: The Development of Advanced Runtime Sound Systems for Games and Extended Reality*, New York, USA: Routledge, doi:<https://doi.org/10.4324/9781351005548>.

Gorzel, M., Allen, A., Kelly, I., Gungormusler, A., Kammerl, J., Yeh, H., and Boland, F. (2019): ‘Efficient Encoding and Decoding of Binaural Sound with Resonance Audio’, in *AES Conference on Immersive and Interactive Audio*, York, UK, 27-29 March: Audio Engineering Society. Paper Number: 68.

Hoene, C. (2023): ‘libmysofa’, URL <https://github.com/hoene/libmysofa>. (Accessed: 29 May 2023).

Horsburgh, A. J., McAlpine, K. B., and Clark, D. F. (2011): ‘A Perspective on the Adoption of Ambisonics for Games’, in *Audio Engineering Society Conference: 41st International Conference: Audio for Games*, London, UK, 2-4 February: Audio Engineering Society. Paper Number: 4-3.

Kronlachner, M. and Zotter, F. (2014): ‘Spatial transformations for the enhancement of Ambisonic recordings’, in *2nd Int. Conf. on Spatial Audio (ICSA)*, Erlangen, Germany, 21-23 February.

Majdak, P., Iwaya, Y., Carpentier, T., Nicol, R., Parmentier, M., Roginska, A., Suzuki, Y., Watanabe, K., Wierstorf, H., Ziegelwanger, H., and Noisternig, M. (2013): ‘Spatially Oriented Format for Acoustics: A Data Exchange Format Representing Head-Related Transfer Functions’, in *Audio Engineering Society Convention 134*, Audio Engineering Society. Paper Number: 8880.

Malham, D. G. (1999): *Higher order Ambisonic systems for the spatialisation of sound*, Ann Arbor, MI: Michigan Publishing, University of Michigan Library.

Raw Material Software Limited (2023): ‘JUCE’, URL <https://juce.com/>. (Accessed: 28 May 2023).

Unity Technologies (2023): ‘Unity’, URL <https://unity.com/>. (Accessed: 28 May 2023).

Zhang, M., Guan, T., Chen, L., Fu, T., Su, D., and Qu, T. (2021): ‘Individualized HRTF-based Binaural Renderer for Higher-Order Ambisonics’, Online, 25-28 May: Audio Engineering Society. Paper Number: 10454.